

Module 3 — PATSTAT Explorer: who is this company, really?

45-minute block · TIP4PATLIBS course material

How to read this document. The running text addresses **you, the participant**. Boxes labelled **Trainer** are for whoever is **running the session**; boxes labelled **Trap** are traps that have caught people before.

Learning objective

I know why one company appears in PATSTAT under dozens of different names, I can consolidate them into one defensible group, and I can state what my consolidation missed.

Prerequisites

- **Module 1** — TIP is running, PATSTAT connects, and you have seen a result table.
- **Module 2** — you know that a query is a set of decisions someone made.

Sub-objectives

By the end you can:

1. **Explain why PATSTAT has no companies** — only names, exactly as typed on each application — and why that is a data-model fact rather than a data-quality problem.
2. **Run a name search and read its hit list** critically: which rows belong to the same group, which do not, and which are probably missing.
3. **Consolidate without double-counting** by counting `docdb_family_id` instead of applications, and apply the sanity check that tells you whether your consolidation did anything.
4. **Reuse a consolidation as a filter** to profile the whole group — where it files, what it works on.
5. **Write down what your method cannot see**, in the four categories the notebook names.

Material

Folder	<code>3_patstat_explorer/</code>
Notebooks	<code>1_applicant-consolidation.ipynb</code> (the method, by hand) · <code>2_explorer-app.ipynb</code> (the same method as an app)
Also	<code>3_explorer-documentation.pdf</code>
Runs on	EPO TIP, PATSTAT PROD
Ships	pre-executed (7 of 7 code cells) — you can read it before you run it

Phase 1 · Introduction (≈ 7 min)

The question this module answers

A client asks: "how big is Siemens Healthineers' patent portfolio?" You run a name search. You get 200 rows. Which one is the answer?

The honest answer is **none of them**, and that is the module.

Put the question to the room and let someone propose "take the biggest row". Then say what that costs: the biggest single spelling of a real corporate group is typically a fraction of the group. Anyone who reports it as the portfolio has undercounted their client's competitor — and has done so in a way that looks like a clean, confident number.

PATSTAT does not know companies. It knows names, one entry per spelling, per country, per subsidiary, per typo. There is no "Siemens Healthineers" record to look up. Somebody has to decide which names are the same organisation, and that somebody is you.

Teaching and learning activity		Time
Opening	Trainer poses the portfolio-size question; room proposes an approach	2 min
Tension	Trainer shows/describes the 200-row hit list — the question has no single-row answer	3 min
Framing	Trainer names the module's claim: consolidation is a judgement, and it has to be written down	2 min

Trainer. Module 2's closing sentence returns here: *an analysis you cannot open up is one you cannot defend*. Module 3 adds the harder half — some of what you open up turns out to be a decision you made, not a fact you found. Say it in those words; it is the hinge of the course.

Phase 2 · Working through (≈ 28 min)

Notebook 1 is five steps and four queries. It ships pre-executed, so you can read ahead — but run it, because step 2 is *yours* to fill in.

Step 1 — Find the names (6 min)

Run the setup cell and step 1. You get one row per name spelling with a count next to it.

Two things about that list:

- It is a **prefix search**. `Healthineers Siemens` would not be found. Pick a search term that sits at the *beginning* of the name.
- **The hit list stops at 200 names**. A very broad term loses its long tail silently.

Trap. Anyone who takes the biggest single row and calls it "the portfolio" has just undercounted the company by a wide margin. That is the whole problem on one screen — and it is the screen most people stop at.

Step 2 — Decide what belongs together (7 min)

This is the step no query can do for you. Look at the hit list and decide: are these names the same organisation?

For `Siemens Healthineers` the answer is easy — every hit belongs to the group, so all of them are kept. Had the search been `Siemens`, the list would also carry Siemens Energy and Siemens Mobility, and those have to be dropped by hand.

Your decision becomes one line of code: the list of names the remaining queries treat as one company.

Trainer. This is the minute to spend. Ask the room whether a **joint venture** counts. Whether a company **acquired in 2019** counts for filings made in 2015. There is no right answer — there is only a *stated* answer, and stating it is what makes the number defensible. Let the disagreement stand; do not resolve it.

Step 3 — Count the group without counting anything twice (7 min)

Many of those names protect **the same inventions**. Adding the step-1 counts up double-counts every invention filed under two spellings.

PATSTAT already solves it: filings that protect one invention share a `docdb_family_id`. Counting families instead of applications is the entire trick.

Trap — Sanity check. The consolidated number must come out **at or below** the naive sum. If the two are *equal*, no invention was filed under more than one spelling — unusual for a real corporate group, and a signal to go back to step 2 and look again.

Step 4 — Profile the group (5 min)

The consolidated name list is now just a filter, and it is reusable. Any question you could ask about one applicant you can now ask about the whole group, with the step-2 decision carried through unchanged. The notebook runs two: **where does it file?** (by filing office) and **what does it work on?** (CPC cut to subclass level, e.g. A61B).

Step 5 — The same method, as an application (3 min)

Open `2_explorer-app.ipynb` and run its single cell. It clones (or updates) the Explorer app from GitHub and starts it. **First launch takes 1–2 minutes** while it downloads and installs; later starts take about ten seconds. To stop it, restart the server.

What you get is the notebook you just ran, wearing a user interface: step 1 fills the hit list, step 2 becomes checkboxes, steps 3 and 4 become the charts. It also carries an **AI Query Generator** backed by a PATSTAT MCP server — you ask in natural language and it writes the SQL.

The point worth taking home. The app sends *the same SQL to the same database* (`PatstatClient(env="PROD")`). Nothing is precomputed, nothing is hidden. You can build a real application on top of PATSTAT on TIP and still see, at any moment, which query produced the number on screen.

Trainer. If the network is slow or the clone fails, do not fight it — the app is a demonstration of a principle you have already taught by hand. Describe it, show the documentation PDF, and move on. Nothing in Phase 3 depends on the app having started.

Step	What you do	What you see	Time
1	Search a company name	Up to 200 spellings, none of them "the answer"	6 min
2	Decide what belongs together	Your judgement, as one line of code	7 min
3	Count families, not applications	A number lower than the naive sum	7 min
4	Profile the group	Where it files, what it works on	5 min
5	Launch the Explorer app	The same method behind a UI — same SQL, same database	3 min

Phase 3 · Learning outcome (≈ 10 min)

What now exists

- One consolidated applicant group — **a named list of spellings you decided belong together.**
- The group's family count, filing offices and technology profile.
- A written note of what your consolidation missed.

That last item is the deliverable that distinguishes a PATLIB from a search engine.

What this can and cannot see

Copy this table into your own working notes; it is the disclaimer that belongs under every portfolio number you hand out.

Missed	A subsidiary whose name does not start with your search term — the prefix search never sees it. Run a second search term for it.
Over-collected	A broad term pulls in unrelated divisions. That is what step 2 is for.
Capped	The hit list stops at 200 names. Very broad terms lose the long tail.
Judgement call	Whether a subsidiary counts as part of the group is a question about the <i>company</i> , not about the data. Decide deliberately — and write down what you decided.

Self-check

1. **Your consolidated family count equals the naive sum of the step-1 rows. Good news?** (*No — suspicious. It means no invention appears under two spellings. Re-check step 2.*)
2. **Why is a prefix search a design decision rather than a bug?** (*Because PATSTAT stores names as typed; any grouping is imposed from outside, and a prefix is a cheap, explainable rule with a known failure mode.*)
3. **A client disputes your number for their competitor.** What do you show them? (*The name list from step 2. The argument is never about the SQL; it is always about which names you kept.*)
4. **Would counting applications ever be right?** (*Yes — when the question is about filing activity or office workload rather than about inventions. State which you used.*)

Transfer to your own work

Consolidate **one organisation you are actually asked about** — ideally one with subsidiaries or a recent acquisition. Save two things: the name list, and one sentence for each of the four rows above saying what it means *for that organisation*.

Trainer. Collect one participant's name list and interrogate it in front of the room for two minutes. Not to catch them out — to demonstrate that the interrogation is survivable when the decision was written down, and not survivable when it was not.

Where this leads

Next	Why
Module 4 — Lead Generation	The same family-counting logic applied to a whole <i>region</i> instead of a single company.

Next	Why
Module 5 — Patent Reports	Applicant consolidation is one section of a landscape report; module 5 builds the rest around it.
Module 6 — IPScore	Module 3 taught you to mark which parts of an answer are judgement. Module 6 makes that marking the output.

Notes for the next revision

- Module 3 is one of the two thin modules (≈ 10 minutes of notebook content). This block works because steps 2 and 5 carry real discussion weight — do not cut the discussion to save time.
- The header of `2_explorer-app.ipynb` contains three typos: "*Integlligence*", "*knowlegde*", and "*TIP4PATLIBs*" (inconsistent capitalisation against the rest of the course). Fix before the workshop — it is the first slide-worthy screen in the module.
- The Explorer app is cloned from a public GitHub repository at run time. Confirm it is reachable from the workshop network **before** 17 September, and have the documentation PDF ready as a fallback.